

Research Paper Assistant – Project Overview

This document summarizes the objective, architecture, implementation steps, and skills involved in building an AI-powered Research Paper Assistant using Retrieval-Augmented Generation (RAG), LangChain, and Vector Databases.

1. Project Objective

Build an AI-powered research assistant that helps users interact with scientific papers more efficiently through semantic search, citation-grounded question answering, paper comparison, and automatic literature review generation. The goal is to reduce the time researchers spend reading and organizing large collections of academic papers while improving information discovery using NLP and retrieval systems.

2. High-Level Workflow

- 1 Users upload one or more research papers in PDF format.
- 2 The system extracts and cleans text from the PDFs.
- 3 Documents are split into smaller chunks for retrieval.
- 4 Each chunk is converted into embeddings using an embedding model.
- 5 Embeddings are stored inside a vector database.
- 6 When the user asks a question, the query is embedded.
- 7 Relevant chunks are retrieved using vector similarity search.
- 8 A Large Language Model generates a grounded response using retrieved context.
- 9 Responses include citations to relevant sections of the papers.

3. Recommended Tech Stack

Category	Tools / Technologies
LLM Framework	LangChain
Vector Database	Qdrant
Backend	FastAPI
Frontend	React / Next.js
PDF Processing	PyMuPDF / Unstructured
Deployment	Docker, AWS
Storage	AWS S3

4. Suggested Development Steps

- 1 Build PDF upload functionality.
- 2 Extract text from uploaded papers.
- 3 Implement chunking and preprocessing.
- 4 Generate embeddings for text chunks.
- 5 Store embeddings inside Qdrant.

- 6 Implement semantic retrieval.
- 7 Connect retrieval results to an LLM using LangChain.
- 8 Add citation-aware prompting.
- 9 Support querying across multiple papers.
- 10 Add literature review and paper comparison features.
- 11 Deploy the system using Docker and cloud infrastructure.

5. Skills Demonstrated

- Retrieval-Augmented Generation (RAG)
- Embeddings and semantic search
- Vector databases
- Natural Language Processing (NLP)
- Backend API development
- Cloud deployment and infrastructure
- Prompt engineering
- Data preprocessing and text chunking
- System architecture and deployment